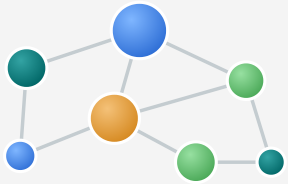

crategraph cheat sheet

Researcher-friendly graph exploration for RO-Crate data. Every filter and transform returns a new `Graph`, so they chain.

General · Edges · Nodes · Operations

THE DATA MODEL

Layer	List	Filter by
Entities (nodes)	`crate.entities`	`where()`, a value
Relationships	`crate.relationships`	`select()`, a type

`where()` filters by a node property. `select()` filters by an edge type.

LOAD A GRAPH

```
from crategraph import Crate

crate = Crate("crates/UMPC")
crate          # Graph(4270, 12601)

Crate(p1, p2)  # several crates
Crate(path, include_root=True)
```

OVERVIEW & ANALYSE

```
crate.summary()    # types + rel counts
crate.profile()    # density, parts
crate.coverage()   # coverage by type
crate.glimpse()    # type-level view
len(crate)         # 4270
crate.most_connected(n=3)
```

RELATIONSHIPS (EDGES)

```
list(crate.relationship_types)
crate.relationship_counts("type")
crate.relationship_records(
    columns=["source", "target"])
crate.select(relationship_types="Primary")
crate.select(
    relationship_types=["Previous", "Related"])
```

EDGES: PATTERN & DERIVE

```
crate.pattern(via="preparedBy")
crate.annotate_relationships(
    pair=lambda r:
        f"{r.source.type}→{r.target.type}"
).relationship_counts("pair")
crate.collapse_edges()  # merge parallels
```

SUBSET THE GRAPH

```
crate.select(entity_types=["Person"])
crate.select(time_range=(1945, 1960))
crate.select(min_connections=20)
crate.exclude(relationship_types="Related")
crate.drop(preparer, property="preparedBy")
crate.subtract(other)
```

SEARCH (FUZZY)

```
crate.search("Elisabet")  # score ⩾ 80
crate.search(
    "Elisabeth", threshold=90, top_n=3)
```

Returns a subgraph; chains with `where` and `expand`.

ENTITIES (NODES)

```
crate.entities          # all nodes
crate.files             # file entities
list(crate.types)       # all type names
crate.get(e.id)         # by id
crate.entity_view(e.id) # live view
e = crate.where(name="…").entities[0]
```

PROPERTIES & COUNTS

```
e.properties            # the data dict
crate.entity_counts("function")
crate.entity_records(
    columns=["id", "label", "type"])
keys = {k for e in crate.entities
          for k in e.properties}
```

FILTER ENTITIES

```
crate.where(function="Lawyer")
crate.where(startDate=(1870, 1900))
crate.annotate_entities(
    is_vic=lambda e:
        e.related("birthState").first("name")
).where(is_vic="Victoria")
```

EXPORT

```
crate.write("out.graphml", format="graphml")
crate.to_networkx()     # NetworkX graph
pd.DataFrame(crate.entity_records())
```

TRANSFORM

```
crate.convert_dates()   # parse dates
crate.convert_dates(start="startDate")
crate.merge_nodes(by="type")
crate.detect_communities(seed=42)
crate.where(function="Lawyer").expand(depth=1)
crate.query("(a:Person)-[:birthPlace]→(b)")
```

FILE TEXT & SEMANTIC

```
len(crate.files)
crate.text_records()    # text per file
crate.build_semantic_index("idx.db")
crate.search(
    "women in science", mode="semantic")
crate.chunk_records("…", k=1)
```

VISUALISE

```
crate.visualise(filepath="net.html")
crate.visualise(renderer="3d")
crate.visualise(
    colour_by="community", size_by="year")
crate.gallery(columns=4)
```

renderer: `2d` (default), `3d`, `svg`, `pyvis`.